

울산_4반_박인우 day1 실습

실습 1. PK = (order_id, prod_id)

order_id	order_dt	cust_id	cust_name	prod_id	prod_name	unit_price	qty
O1001	2024-06-01	C01	김하늘	P10	노트북	1200000	1
O1001	2024-06-01	C01	김하늘	P22	마우스	25000	2
O1002	2024-06-03	C02	이준호	P10	노트북	1200000	1

실습 과제 — 이 표를 3NF로

1. 각 컬럼의 함수 종속 식별

order_id → order_dt, cust_id

cust_id → cust_name

prod_id → prod_name, unit_price

(order_id, prod_id) → qty

prod_name → unit_price은 같은 상품명을 가진 다른 상품이 존재할 수도 있기에
안됨

qty는 어떤주문에서 어떤 상품을 주문했는지가 있어야 결정

2. 1NF→2NF→3NF 단계로 분해

1NF : 각 칸에 하나의 값만 들어가고, 반복되는 컬럼 묶음이 없어야한다 → 표를
보면 만족

2NF : 복합 기본키의 일부에만 종속되는 컬럼을 분리

order_id에만 종속 : order_dt, cust_id, cust_name

prod_id에만 종속 : prod_name, unit_price

(order_id, prod_id)에 종속 → qty

주문(

order_id PK,
order_dt,
cust_id,
cust_name

)

상품(

prod_id PK,

```
        prod_name,  
        unit_price  
    )
```

```
주문상세(  
    order_id PK, FK,  
    prod_id PK, FK,  
    qty  
)
```

3NF : 이행 함수 종속 제거($\text{cust_id} \rightarrow \text{cust_name}$)

```
고객 (  
    cust_id PK,  
    cust_name  
)  
주문(  
    order_id PK,  
    order_dt,  
    cust_id FK  
)
```

3. 최종 테이블·PK·FK 정리

```
고객(  
    cust_id PK,  
    cust_name  
)
```

```
주문(  
    order_id PK,  
    order_dt,  
    cust_id FK  
)
```

```
상품(  
    prod_id PK,  
    prod_name,  
    unit_price  
)
```

```
주문상세(  
    order_id PK, FK,  
    prod_id PK, FK,  
    qty  
)
```

고객 1 —— N 주문 # 고객 한명은 주문을 여러번 가능
 주문 1 —— N 주문상세 # 주문 하나에는 여러상품 가능
 상품 1 —— N 주문상세 # 상품 하나에는 여러 주문에서 판매 가능

추가. 실제 dbeaver 이용해보기

```

● CREATE TABLE order_detail_raw (
  order_id VARCHAR(10) NOT NULL,
  order_dt DATE NOT NULL,
  cust_id VARCHAR(10) NOT NULL,
  cust_name VARCHAR(50) NOT NULL,
  prod_id VARCHAR(10) NOT NULL,
  prod_name VARCHAR(50) NOT NULL,
  unit_price INTEGER NOT NULL,
  qty INTEGER NOT NULL,
  PRIMARY KEY (order_id, prod_id)
);

```

```

● INSERT INTO order_detail_raw (
  order_id,
  order_dt,
  cust_id,
  cust_name,
  prod_id,
  prod_name,
  unit_price,
  qty
)
VALUES
  ('O1001', DATE '2024-06-01', 'C01', '김하늘', 'P10', '노트북', 1200000, 1),
  ('O1001', DATE '2024-06-01', 'C01', '김하늘', 'P22', '마우스', 25000, 2),
  ('O1002', DATE '2024-06-03', 'C02', '이준호', 'P10', '노트북', 1200000, 1);

```

	order_id	order_dt	cust_id	cust_name	prod_id	prod_name	unit_price	qty
1	O1001	2024-06-01	C01	김하늘	P10	노트북	1,200,000	1
2	O1001	2024-06-01	C01	김하늘	P22	마우스	25,000	2
3	O1002	2024-06-03	C02	이준호	P10	노트북	1,200,000	1

lab 스키마

```

● CREATE TABLE customer (
  cust_id VARCHAR(10) PRIMARY KEY,
  cust_name VARCHAR(50) NOT NULL
);

● CREATE TABLE product (
  prod_id VARCHAR(10) PRIMARY KEY,
  prod_name VARCHAR(50) NOT NULL,
  unit_price INTEGER NOT NULL CHECK (unit_price >= 0)
);

● CREATE TABLE orders (
  order_id VARCHAR(10) PRIMARY KEY,
  order_dt DATE NOT NULL,
  cust_id VARCHAR(10) NOT NULL,
  FOREIGN KEY (cust_id)
    REFERENCES customer(cust_id)
);

● CREATE TABLE order_item (
  order_id VARCHAR(10) NOT NULL,
  prod_id VARCHAR(10) NOT NULL,
  qty INTEGER NOT NULL CHECK (qty > 0),
  PRIMARY KEY (order_id, prod_id),
  FOREIGN KEY (order_id)
    REFERENCES orders(order_id),
  FOREIGN KEY (prod_id)
    REFERENCES product(prod_id)
);

```

```

● INSERT INTO lab.customer (cust_id, cust_name)
SELECT DISTINCT
  cust_id,
  cust_name
FROM public.order_detail_raw;

● INSERT INTO lab.product (prod_id, prod_name, unit_price)
SELECT DISTINCT
  prod_id,
  prod_name,
  unit_price
FROM public.order_detail_raw;

● INSERT INTO lab.orders (order_id, order_dt, cust_id)
SELECT DISTINCT
  order_id,
  order_dt,
  cust_id
FROM public.order_detail_raw;

● INSERT INTO lab.order_item (order_id, prod_id, qty)
SELECT
  order_id,
  prod_id,
  qty
FROM public.order_detail_raw;

```

lab

Tables

customer

8K

order_item

8K

orders

8K

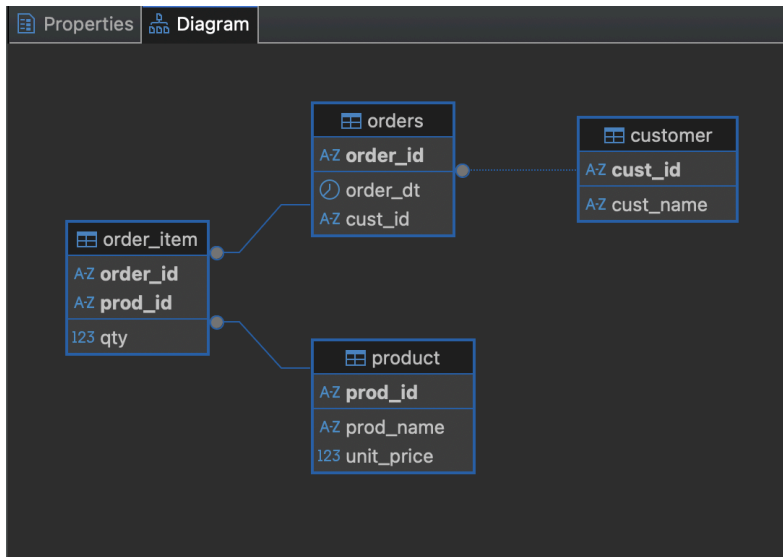
product

8K

	cust_id	cust_name
1	C01	김하늘
2	C02	이준호

	order_id	prod_id	qty
1	O1001	P10	1
2	O1001	P22	2
3	O1002	P10	1

	AZ prod_id	AZ prod_name	123 unit_price
1	P10	노트북	1,200,000
2	P22	마우스	25,000



실습 2 . 단일 테이블 조회

```

select
  cust_id ,
  name,
  birth_ymd,
  grade,
  join_dt ,
  coalesce(region,'미상') as region,  # null → '미상'
  extract (year from age(birth_ymd)) as age,
  case
    when extract (year from age(birth_ymd)) <30 then '청년'
    when extract (year from age(birth_ymd)) <50 then '중년'
    else '장년'
  end as age_type,
from customer

where
  grade in ('GOLD', 'VIP')
  and join_dt >= Date '2020-01-01'

order by join_dt asc limit 15;
  
```

```

select
  ● cust_id ,
    name,
    birth_ymd,
    grade,
    join_dt ,
    coalesce(region,'미상') as region,
    extract (year from age(birth_ymd)) as age,

  ● case
    when extract (year from age(birth_ymd)) <30 then '청년'
    when extract (year from age(birth_ymd)) <50 then '중년'
    else '장년'
  end as age_type]

from customer

● where
  grade in ('GOLD', 'VIP')
  and join_dt >= Date '2020-01-01'

order by join_dt asc limit 15;

```

실행 결과

	123	cust_id	AZ name	birth_ymd	AZ grade	join_dt	AZ region	123 age	AZ age_type
1		211	고객211	2014-11-13	GOLD	2022-01-05	부산	11	청년
2		198	고객198	1993-09-30	GOLD	2022-01-06	강원	32	중년
3		155	고객155	2016-07-10	GOLD	2022-01-07	강원	10	청년
4		15	고객15	1995-09-23	GOLD	2022-01-12	대전	30	중년
5		320	고객320	1977-09-26	GOLD	2022-01-12	서울	48	중년
6		227	고객227	2010-06-03	GOLD	2022-01-13	부산	16	청년
7		115	고객115	1979-12-15	VIP	2022-01-16	대전	46	중년
8		216	고객216	2006-03-15	GOLD	2022-01-16	강원	20	청년
9		196	고객196	2015-04-12	GOLD	2022-01-29	대전	11	청년
10		430	고객430	2002-08-01	GOLD	2022-01-30	경기	23	청년
11		309	고객309	2015-02-12	GOLD	2022-01-31	대구	11	청년
12		225	고객225	1977-08-05	GOLD	2022-02-01	대구	48	중년
13		243	고객243	2008-02-26	GOLD	2022-02-04	대전	18	청년
14		209	고객209	1996-10-01	GOLD	2022-02-07	경기	29	청년
15		453	고객453	2007-12-31	GOLD	2022-02-09	광주	18	청년

개인 정리(주의점, 틀렸던 부분)

and join_dt >= Date '2020-01-01' (처음 작성: and join_dt >='2020-01-01'),
더 권장되는 작성 방식

case

```

when extract (year from age(birth_ymd)) <30 then '청년'
when extract (year from age(birth_ymd)) <50 then '중년'
else '장년'

```

(처음 작성 :

case

```

when age<30 then '청년'
when 30 < age <50 then '중년'
else '장년')

```

같은 select절에서 생성한 별칭은 다른데서 참조를 못함.

또한 SQL에서는 30<age<50같은 연속 비교를 사용할 수 없으므로 age>30 and age <50 처럼 작성해야 한다. 다만 case는 위에서부터 검사하고 처음 참인 조건에서 종료되므로, 첫 번째 조건에서 30세 미만이 이미 처리되어 두 번째 조건은 <50만 작성해도 30~49세를 의미한다.